

Practical 3

Aim: To implement a Word Count / Frequency Program using MapReduce in Hadoop and analyze word frequencies in a text dataset stored in HDFS.

Problem definition:

A digital publishing company stores thousands of news articles and blog posts in the Hadoop Distributed File System (HDFS). The editorial team wants to identify the most frequently used words across their content to understand trending topics and optimize content recommendations.

As a Big Data Engineer, the task is to develop a MapReduce application that processes text data and generates word frequency statistics.

Objectives:

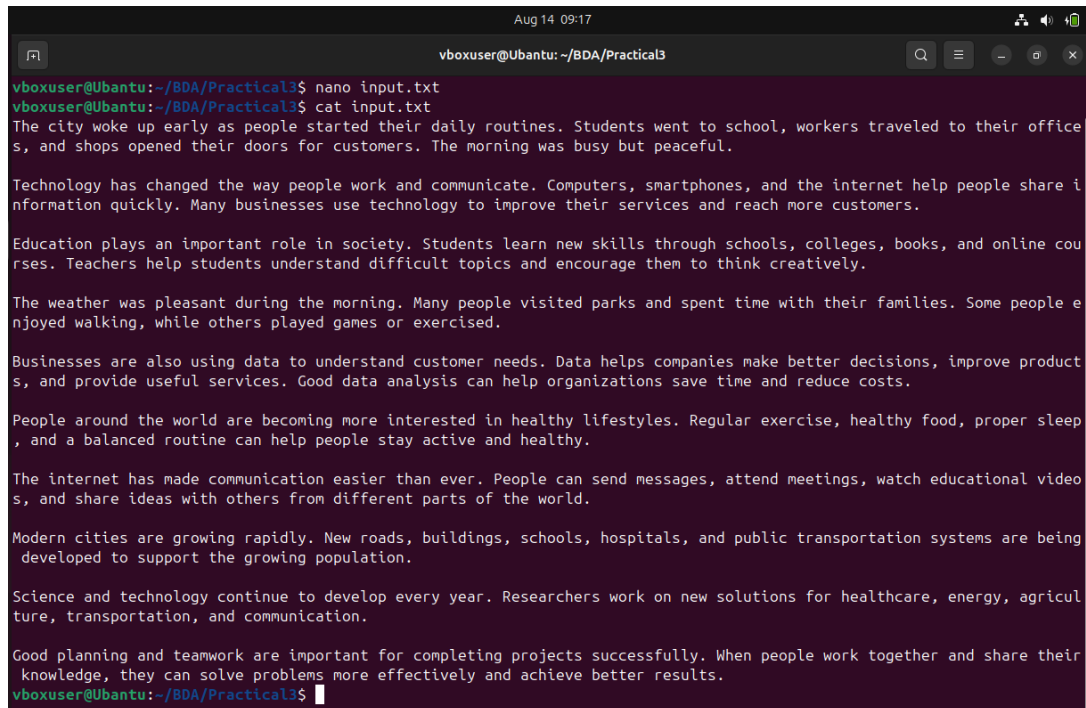
- Configure the Hadoop environment and HDFS.
- Create and upload a text dataset to HDFS.
- Implement a **Mapper** to generate (word, 1) key-value pairs.
- Implement a **Reducer** to calculate word frequencies.
- Execute the MapReduce job on Hadoop.
- Store the output in HDFS.
- Find the top 10 most frequently occurring words.
- Analyze the performance for different dataset sizes.

Observation / Output

Step 1: Create the Input Dataset

Open the terminal and create the input file:

nano input.txt

A terminal window titled 'vboxuser@Ubuntu: ~/BDA/Practical3' with a search bar and window controls. The terminal shows the user creating 'input.txt' with 'nano' and then displaying its contents with 'cat'. The file contains several paragraphs of text about daily routines, technology, education, weather, business, health, internet, city growth, and science.

```
Aug 14 09:17
vboxuser@Ubuntu: ~/BDA/Practical3
vboxuser@Ubuntu:~/BDA/Practical3$ nano input.txt
vboxuser@Ubuntu:~/BDA/Practical3$ cat input.txt
The city woke up early as people started their daily routines. Students went to school, workers traveled to their office
s, and shops opened their doors for customers. The morning was busy but peaceful.

Technology has changed the way people work and communicate. Computers, smartphones, and the internet help people share i
nformation quickly. Many businesses use technology to improve their services and reach more customers.

Education plays an important role in society. Students learn new skills through schools, colleges, books, and online cou
rses. Teachers help students understand difficult topics and encourage them to think creatively.

The weather was pleasant during the morning. Many people visited parks and spent time with their families. Some people e
njoyed walking, while others played games or exercised.

Businesses are also using data to understand customer needs. Data helps companies make better decisions, improve product
s, and provide useful services. Good data analysis can help organizations save time and reduce costs.

People around the world are becoming more interested in healthy lifestyles. Regular exercise, healthy food, proper sleep
, and a balanced routine can help people stay active and healthy.

The internet has made communication easier than ever. People can send messages, attend meetings, watch educational video
s, and share ideas with others from different parts of the world.

Modern cities are growing rapidly. New roads, buildings, schools, hospitals, and public transportation systems are being
developed to support the growing population.

Science and technology continue to develop every year. Researchers work on new solutions for healthcare, energy, agricul
ture, transportation, and communication.

Good planning and teamwork are important for completing projects successfully. When people work together and share their
knowledge, they can solve problems more effectively and achieve better results.
vboxuser@Ubuntu:~/BDA/Practical3$
```

Figure 1:shows the created input.txt and its contents.

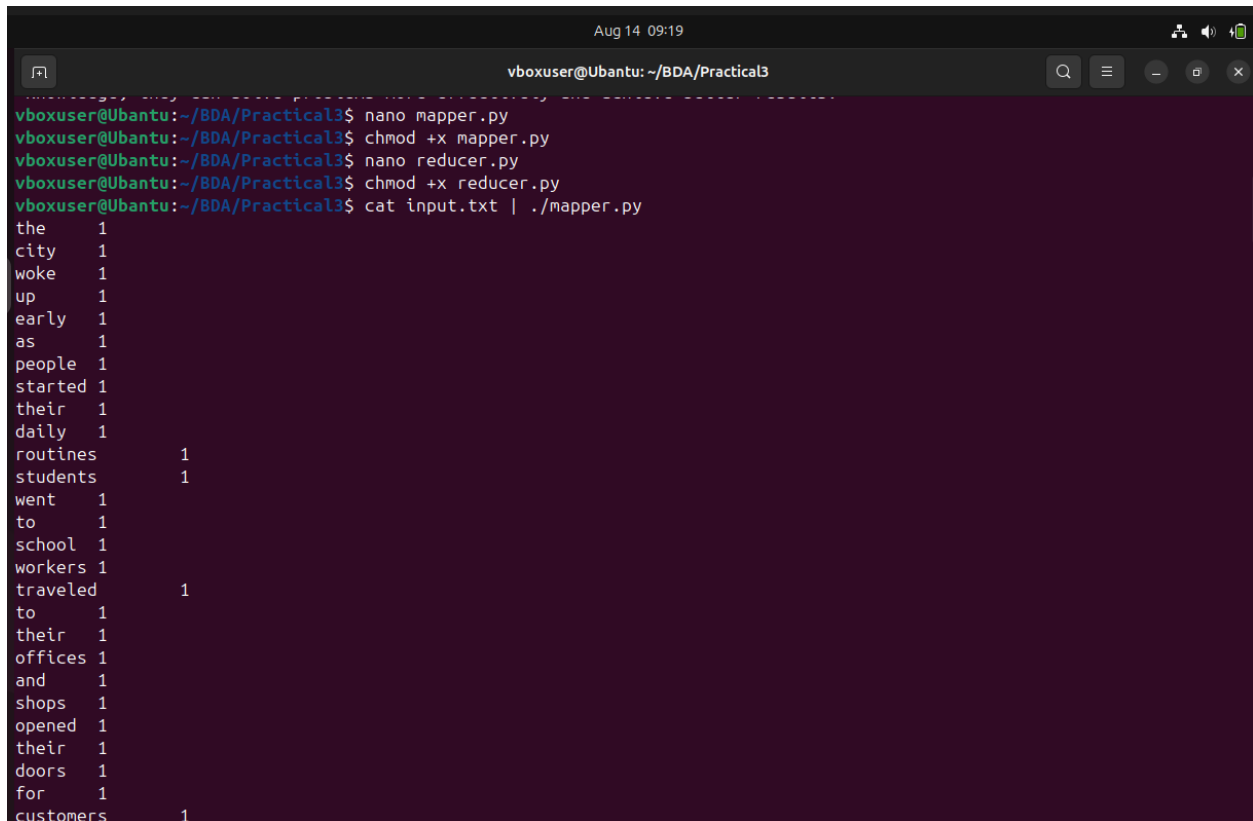
Step 2: Create the Mapper and Reducer Programs

Create the mapper file:

```
nano mapper.py
```

Create the reducer file:

```
nano reducer.py
```

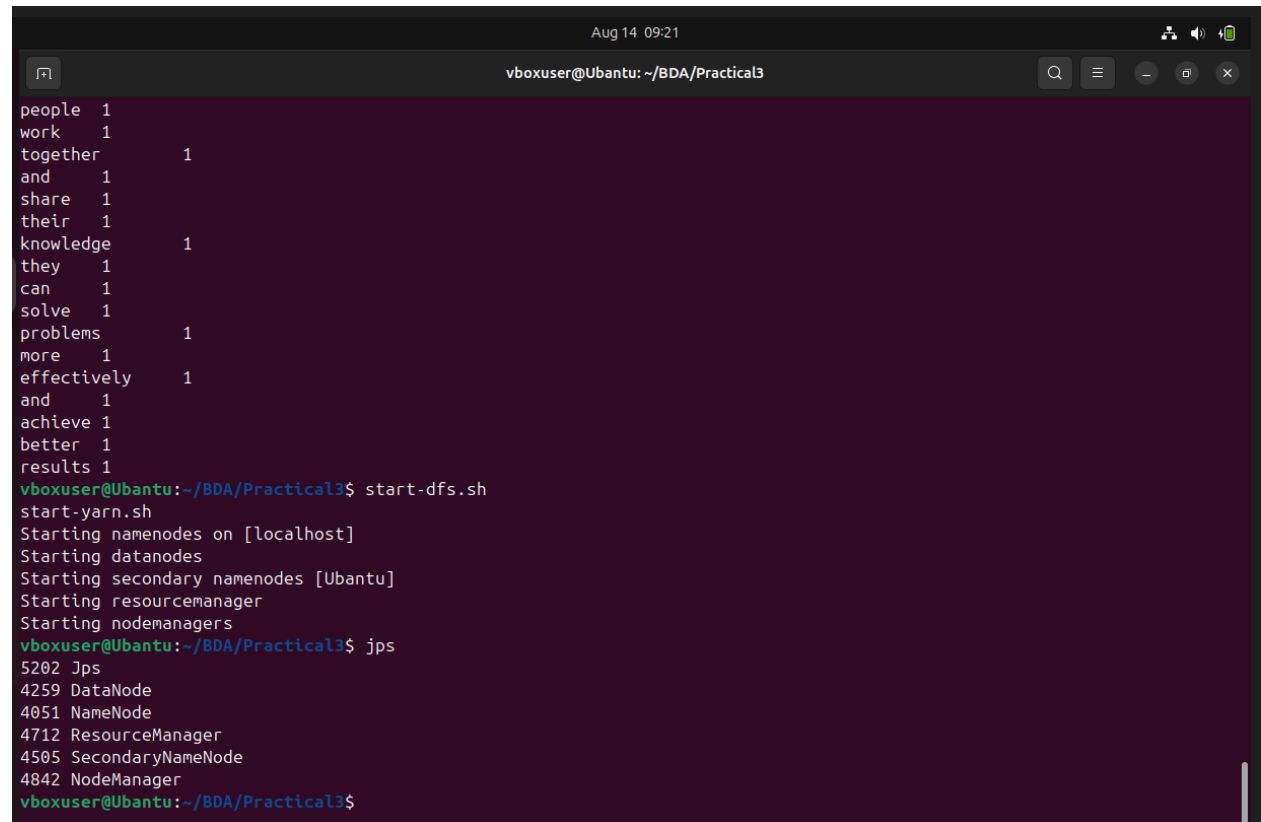


```
vboxuser@Ubuntu:~/BDA/Practical3$ nano mapper.py
vboxuser@Ubuntu:~/BDA/Practical3$ chmod +x mapper.py
vboxuser@Ubuntu:~/BDA/Practical3$ nano reducer.py
vboxuser@Ubuntu:~/BDA/Practical3$ chmod +x reducer.py
vboxuser@Ubuntu:~/BDA/Practical3$ cat input.txt | ./mapper.py
the 1
city 1
woke 1
up 1
early 1
as 1
people 1
started 1
their 1
daily 1
routines 1
students 1
went 1
to 1
school 1
workers 1
traveled 1
to 1
their 1
offices 1
and 1
shops 1
opened 1
their 1
doors 1
for 1
customers 1
```

Figure 2: shows the mapper output.

Step 3: Start Hadoop Services

Start the Hadoop Distributed File System and YARN services:

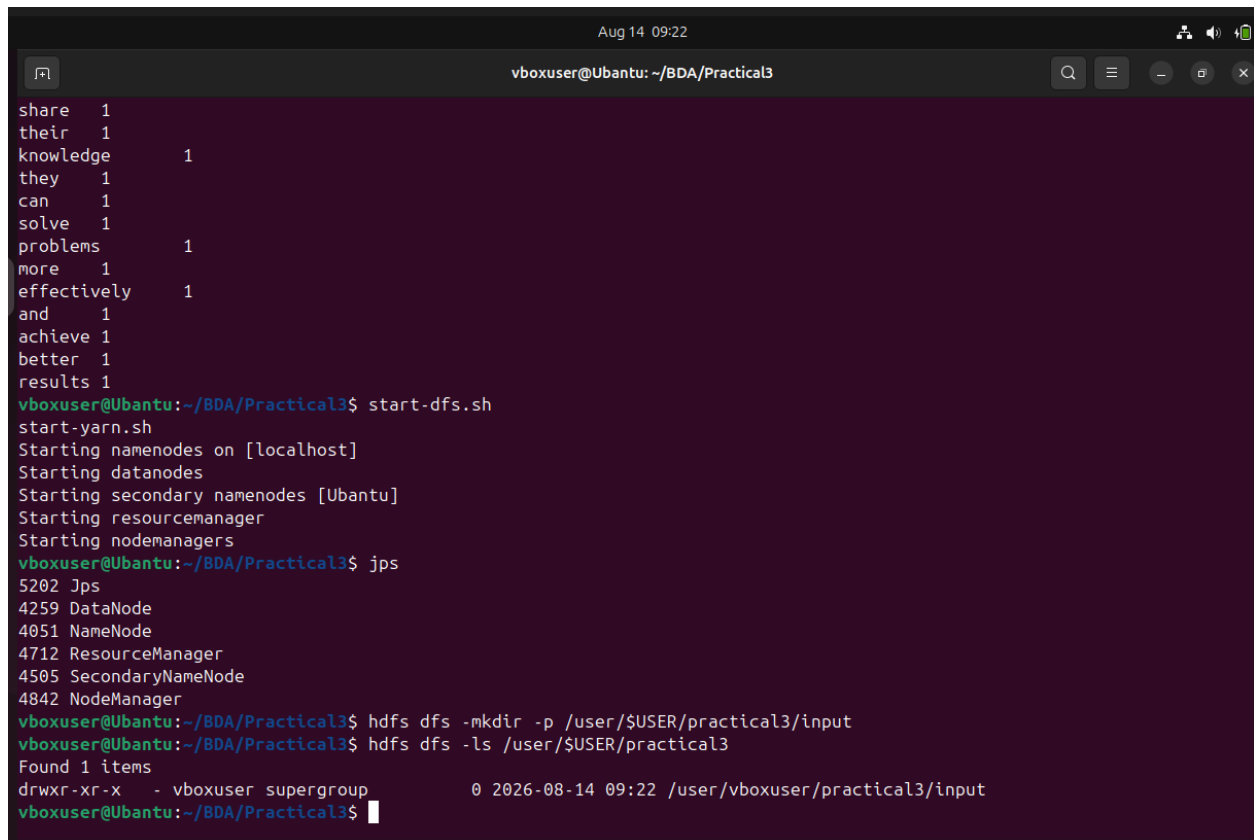
A terminal window titled 'vboxuser@Ubuntu: ~/BDA/Practical3' with a timestamp of 'Aug 14 09:21'. The terminal shows a list of words and their counts, followed by the execution of 'start-dfs.sh' and 'start-yarn.sh'. It then shows the execution of 'jps' which lists the running Hadoop processes: Jps, DataNode, NameNode, ResourceManager, SecondaryNameNode, and NodeManager.

```
people 1
work 1
together 1
and 1
share 1
their 1
knowledge 1
they 1
can 1
solve 1
problems 1
more 1
effectively 1
and 1
achieve 1
better 1
results 1
vboxuser@Ubuntu:~/BDA/Practical3$ start-dfs.sh
start-yarn.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Ubuntu]
Starting resourcemanager
Starting nodemanagers
vboxuser@Ubuntu:~/BDA/Practical3$ jps
5202 Jps
4259 DataNode
4051 NameNode
4712 ResourceManager
4505 SecondaryNameNode
4842 NodeManager
vboxuser@Ubuntu:~/BDA/Practical3$
```

Figure 3: shows the Hadoop services being started and verified using jps.

Step 4: Create the HDFS Input Directory

Create an input directory in HDFS:

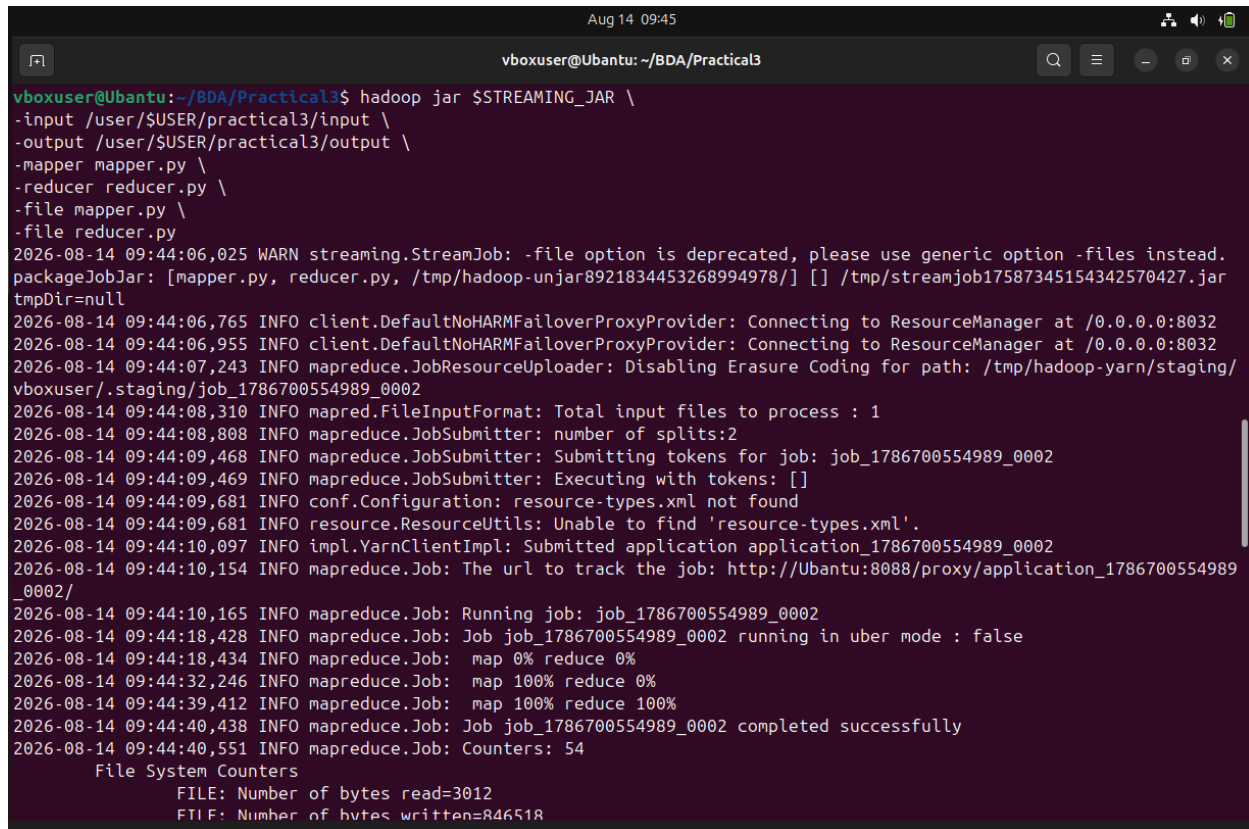
A terminal window titled 'vboxuser@Ubuntu: ~/BDA/Practical3' with a timestamp of 'Aug 14 09:22'. The terminal shows a word count of a text file, followed by running 'start-dfs.sh' and 'start-yarn.sh'. It then lists running processes with 'jps', showing NameNode, ResourceManager, SecondaryNameNode, and NodeManager. Finally, it runs 'hdfs dfs -mkdir -p /user/\$USER/practical3/input' and 'hdfs dfs -ls /user/\$USER/practical3', showing the directory was created and contains one item.

```
share 1
their 1
knowledge 1
they 1
can 1
solve 1
problems 1
more 1
effectively 1
and 1
achieve 1
better 1
results 1
vboxuser@Ubuntu:~/BDA/Practical3$ start-dfs.sh
start-yarn.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Ubuntu]
Starting resourcemanager
Starting nodemanagers
vboxuser@Ubuntu:~/BDA/Practical3$ jps
5202 Jps
4259 DataNode
4051 NameNode
4712 ResourceManager
4505 SecondaryNameNode
4842 NodeManager
vboxuser@Ubuntu:~/BDA/Practical3$ hdfs dfs -mkdir -p /user/$USER/practical3/input
vboxuser@Ubuntu:~/BDA/Practical3$ hdfs dfs -ls /user/$USER/practical3
Found 1 items
drwxr-xr-x - vboxuser supergroup 0 2026-08-14 09:22 /user/vboxuser/practical3/input
vboxuser@Ubuntu:~/BDA/Practical3$
```

Figure 4 shows the HDFS input directory being created and checked.

Step 5: Run the Hadoop Streaming MapReduce Job

Run the MapReduce word-count job using Hadoop Streaming:

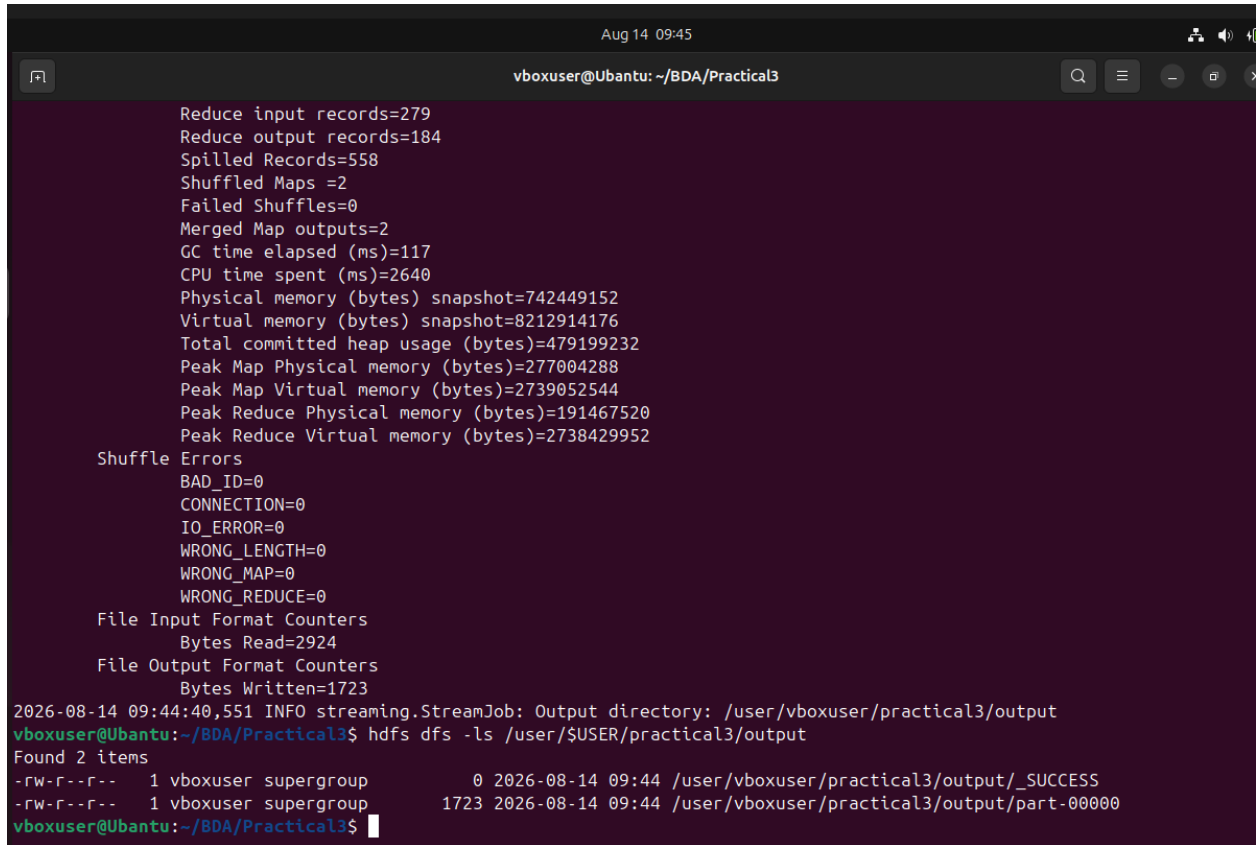
A terminal window titled 'vboxuser@Ubuntu: ~/BDA/Practical3' showing the execution of a Hadoop Streaming MapReduce job. The user runs 'hadoop jar \$STREAMING_JAR \', followed by several options: '-input /user/\$USER/practical3/input \', '-output /user/\$USER/practical3/output \', '-mapper mapper.py \', '-reducer reducer.py \', '-file mapper.py \', and '-file reducer.py'. The output shows various log messages from the Hadoop framework, including warnings about deprecated options and information about job submission and progress. The job is identified as 'job_1786700554989_0002'. The progress shows 'map 100% reduce 0%' and then 'map 100% reduce 100%'. The final status is 'Job job_1786700554989_0002 completed successfully'. At the bottom, file system counters are displayed: 'FILE: Number of bytes read=3012' and 'FILE: Number of bytes written=846518'.

```
vboxuser@Ubuntu:~/BDA/Practical3$ hadoop jar $STREAMING_JAR \
-input /user/$USER/practical3/input \
-output /user/$USER/practical3/output \
-mapper mapper.py \
-reducer reducer.py \
-file mapper.py \
-file reducer.py
2026-08-14 09:44:06,025 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.
packageJobJar: [mapper.py, reducer.py, /tmp/hadoop-unjar8921834453268994978/] [] /tmp/streamjob17587345154342570427.jar
tmpDir=null
2026-08-14 09:44:06,765 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-08-14 09:44:06,955 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-08-14 09:44:07,243 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/
vboxuser/.staging/job_1786700554989_0002
2026-08-14 09:44:08,310 INFO mapred.FileInputFormat: Total input files to process : 1
2026-08-14 09:44:08,808 INFO mapreduce.JobSubmitter: number of splits:2
2026-08-14 09:44:09,468 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1786700554989_0002
2026-08-14 09:44:09,469 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-08-14 09:44:09,681 INFO conf.Configuration: resource-types.xml not found
2026-08-14 09:44:09,681 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-08-14 09:44:10,097 INFO impl.YarnClientImpl: Submitted application application_1786700554989_0002
2026-08-14 09:44:10,154 INFO mapreduce.Job: The url to track the job: http://Ubuntu:8088/proxy/application_1786700554989_
_0002/
2026-08-14 09:44:10,165 INFO mapreduce.Job: Running job: job_1786700554989_0002
2026-08-14 09:44:18,428 INFO mapreduce.Job: Job job_1786700554989_0002 running in uber mode : false
2026-08-14 09:44:18,434 INFO mapreduce.Job: map 0% reduce 0%
2026-08-14 09:44:32,246 INFO mapreduce.Job: map 100% reduce 0%
2026-08-14 09:44:39,412 INFO mapreduce.Job: map 100% reduce 100%
2026-08-14 09:44:40,438 INFO mapreduce.Job: Job job_1786700554989_0002 completed successfully
2026-08-14 09:44:40,551 INFO mapreduce.Job: Counters: 54
File System Counters
FILE: Number of bytes read=3012
FILE: Number of bytes written=846518
```

Figure5:shows the Hadoop Streaming MapReduce job running successfully.

Step 6: Check the MapReduce Output

After the job finishes, check the output directory:

A terminal window titled 'vboxuser@Ubuntu: ~/BDA/Practical3' showing the output of a MapReduce job. The output includes statistics for the reduce phase, shuffle errors, and file input/output counters. It also shows the command 'hdfs dfs -ls /user/\$USER/practical3/output' and its output, which lists two files: '_SUCCESS' and 'part-00000'.

```
Aug 14 09:45
vboxuser@Ubuntu: ~/BDA/Practical3

Reduce input records=279
Reduce output records=184
Spilled Records=558
Shuffled Maps =2
Failed Shuffles=0
Merged Map outputs=2
GC time elapsed (ms)=117
CPU time spent (ms)=2640
Physical memory (bytes) snapshot=742449152
Virtual memory (bytes) snapshot=8212914176
Total committed heap usage (bytes)=479199232
Peak Map Physical memory (bytes)=277004288
Peak Map Virtual memory (bytes)=2739052544
Peak Reduce Physical memory (bytes)=191467520
Peak Reduce Virtual memory (bytes)=2738429952

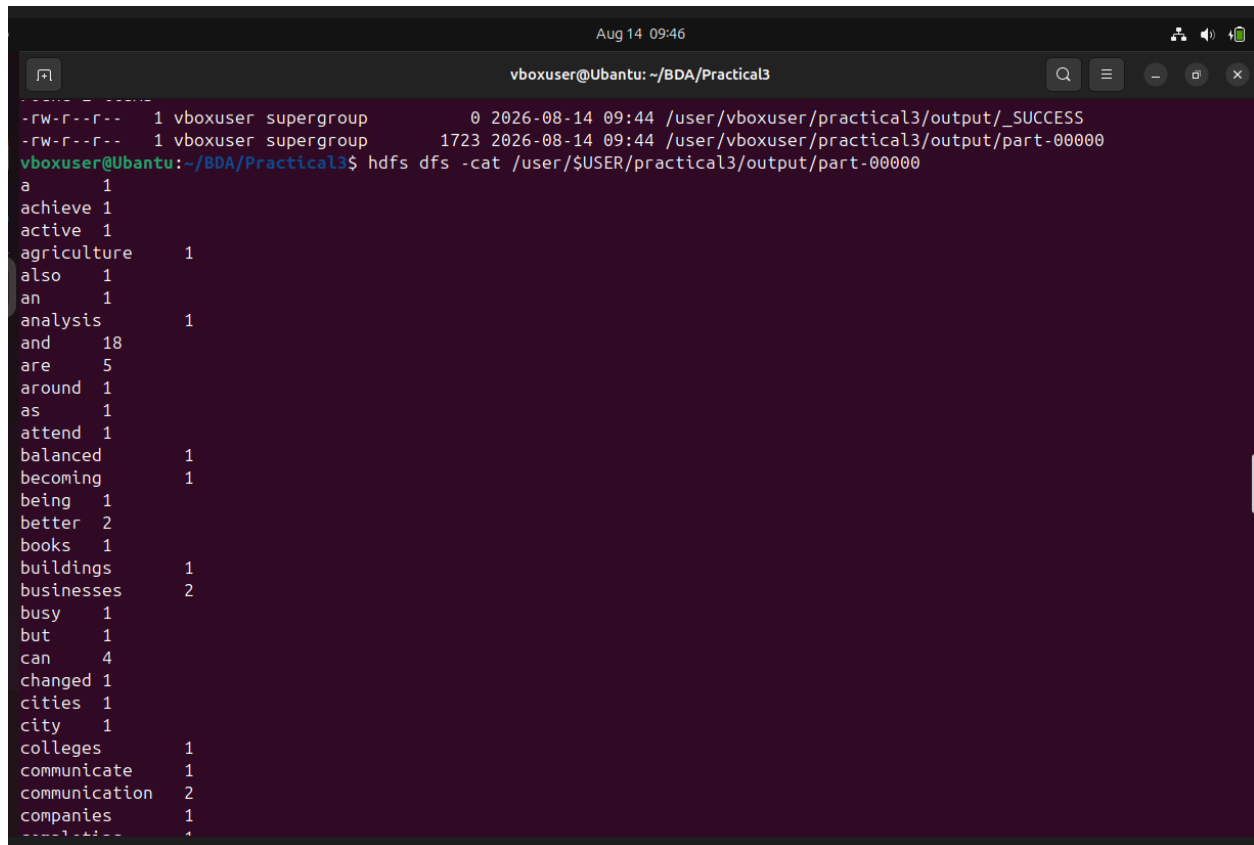
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0

File Input Format Counters
Bytes Read=2924
File Output Format Counters
Bytes Written=1723
2026-08-14 09:44:40,551 INFO streaming.StreamJob: Output directory: /user/vboxuser/practical3/output
vboxuser@Ubuntu:~/BDA/Practical3$ hdfs dfs -ls /user/$USER/practical3/output
Found 2 items
-rw-r--r-- 1 vboxuser supergroup 0 2026-08-14 09:44 /user/vboxuser/practical3/output/_SUCCESS
-rw-r--r-- 1 vboxuser supergroup 1723 2026-08-14 09:44 /user/vboxuser/practical3/output/part-00000
vboxuser@Ubuntu:~/BDA/Practical3$
```

Figure 6 shows the successful MapReduce output files.

Step 7: Display the Word Count Results

Display the contents of the output file:

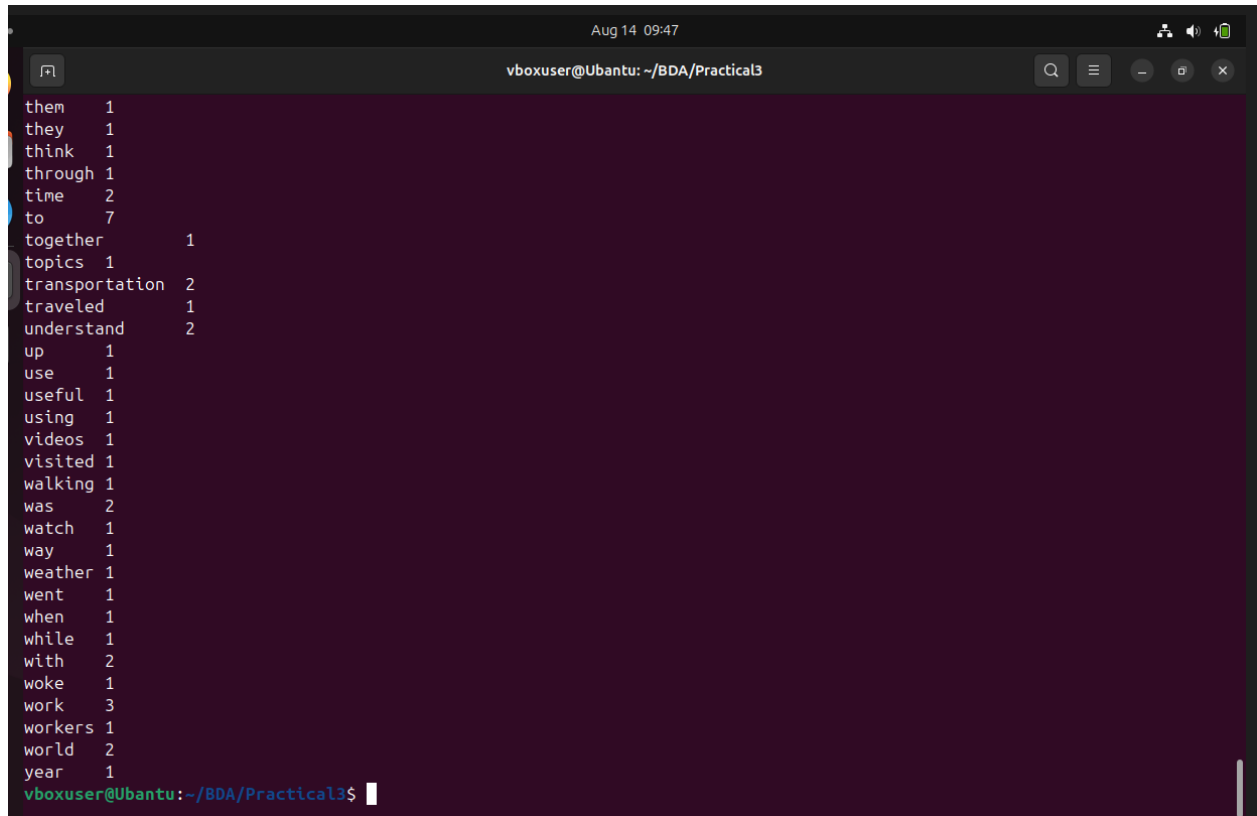
A terminal window titled 'vboxuser@Ubuntu: ~/BDA/Practical3' with a timestamp 'Aug 14 09:46'. It shows the execution of 'hdfs dfs -cat /user/\$USER/practical3/output/part-00000' and the resulting word count output. The output lists words and their counts, such as 'a 1', 'achieve 1', 'active 1', 'agriculture 1', 'also 1', 'an 1', 'analysis 1', 'and 18', 'are 5', 'around 1', 'as 1', 'attend 1', 'balanced 1', 'becoming 1', 'being 1', 'better 2', 'books 1', 'buildings 1', 'businesses 2', 'busy 1', 'but 1', 'can 4', 'changed 1', 'cities 1', 'city 1', 'colleges 1', 'communicate 1', 'communication 2', and 'companies 1'.

```
Aug 14 09:46
vboxuser@Ubuntu: ~/BDA/Practical3
-rw-r--r-- 1 vboxuser supergroup 0 2026-08-14 09:44 /user/vboxuser/practical3/output/_SUCCESS
-rw-r--r-- 1 vboxuser supergroup 1723 2026-08-14 09:44 /user/vboxuser/practical3/output/part-00000
vboxuser@Ubuntu:~/BDA/Practical3$ hdfs dfs -cat /user/$USER/practical3/output/part-00000
a 1
achieve 1
active 1
agriculture 1
also 1
an 1
analysis 1
and 18
are 5
around 1
as 1
attend 1
balanced 1
becoming 1
being 1
better 2
books 1
buildings 1
businesses 2
busy 1
but 1
can 4
changed 1
cities 1
city 1
colleges 1
communicate 1
communication 2
companies 1
-----
```

Figure 7 shows the beginning of the final word-count output.

Verify the Complete Output:

Scroll through the output to verify the remaining word frequencies.

A terminal window titled 'vboxuser@Ubuntu: ~/BDA/Practical3' showing the final section of word-count results. The output lists words and their frequencies, such as 'them 1', 'they 1', 'think 1', 'through 1', 'time 2', 'to 7', 'together 1', 'topics 1', 'transportation 2', 'traveled 1', 'understand 2', 'up 1', 'use 1', 'useful 1', 'using 1', 'videos 1', 'visited 1', 'walking 1', 'was 2', 'watch 1', 'way 1', 'weather 1', 'went 1', 'when 1', 'while 1', 'with 2', 'woke 1', 'work 3', 'workers 1', 'world 2', and 'year 1'. The prompt 'vboxuser@Ubuntu: ~/BDA/Practical3\$' is visible at the bottom.

```
Aug 14 09:47
vboxuser@Ubuntu: ~/BDA/Practical3
them 1
they 1
think 1
through 1
time 2
to 7
together 1
topics 1
transportation 2
traveled 1
understand 2
up 1
use 1
useful 1
using 1
videos 1
visited 1
walking 1
was 2
watch 1
way 1
weather 1
went 1
when 1
while 1
with 2
woke 1
work 3
workers 1
world 2
year 1
vboxuser@Ubuntu: ~/BDA/Practical3$
```

Figure 7 shows the final section of the word-count results.

Key Questions / Analysis

Q1. What is the role of Mapper and Reducer in the Word Count application?

Answer:

The Mapper reads the input data and generates intermediate key-value pairs in the form (word, 1). The Reducer receives the grouped words and adds their values to calculate the total frequency of each word.

Q2. How are intermediate key-value pairs generated and processed?

Answer:

The Mapper splits each line of text into individual words and generates (word, 1) for every word. Hadoop then performs Shuffle and Sort, which groups identical words together. The

Reducer processes these groups and calculates their total frequency.

Example:

Mapper:

hello → 1

hello → 1

data → 1

Shuffle:

hello → [1,1]

data → [1]

Reducer:

hello → 2

data → 1

Q3. How does HDFS store and distribute the input dataset?**Answer:**

HDFS divides large files into blocks and stores these blocks across different DataNodes. It also maintains replicas of the blocks to provide fault tolerance and reliable data storage.

Q4. What advantages does MapReduce offer for processing large datasets?**Answer:**

- Processes large amounts of data efficiently.
- Supports parallel processing.
- Provides fault tolerance.
- Works with distributed storage such as HDFS.
- Can scale across multiple machines.
- Automatically handles tasks such as data distribution and grouping.

Q5. How does the execution time vary with dataset size?**Answer:**

Generally, execution time increases as the dataset size increases. In our experiment, the large dataset took 4.364 seconds, compared with 3.363 seconds for the small dataset. The small difference between small and medium datasets is due to Hadoop's startup and job-processing overhead.

Conclusion:

The Word Count application was successfully implemented using Hadoop MapReduce. The Mapper generated intermediate (word, 1) pairs, while the Reducer calculated the total frequency of each word. The input and output data were stored in HDFS. The experiment also demonstrated the effect of dataset size on MapReduce execution time.